

ARQUITETURA E ESTRUTURA

Sistema Web de Controle de Estoque da Revisão

Registro das decisões técnicas, estruturais, de segurança, auditoria e implantação

Projeto	Sistema de Controle do Estoque da Revisão
Contexto	Projeto de Extensão — Fábrica de Software
Versão do documento	1.0
Data de registro	30 de agosto de 2026

Objetivo deste documento

Consolidar as decisões arquiteturais tomadas para que o desenvolvimento mantenha uma base única, simples, segura e profissional, preparada tanto para execução local quanto para implantação futura em servidor corporativo ou nuvem.

Princípio central

Construir apenas a complexidade necessária agora, preservando uma estrutura que possa crescer sem exigir reescrita do núcleo do sistema.

1. Resumo das decisões

Decisão	Definição
Arquitetura	Monólito modular, com frontend, backend e persistência separados.
Frontend	React + TypeScript + Vite.
Backend	Node.js + TypeScript + NestJS.
API	REST/JSON, versionada a partir de /api/v1.
Banco	PostgreSQL relacional, banco principal único.
Segurança	Autenticação segura, hash de senha com salt, autorização no backend, HTTPS e segredos externos ao código.
Rastreabilidade	Histórico de negócio, auditoria e logs técnicos tratados como conceitos distintos.
Deploy	Execução direta na máquina durante desenvolvimento; Docker/Docker Compose + reverse proxy em produção.
Escalabilidade	Preparação para servidor da empresa ou cloud sem acoplamento da regra de negócio à infraestrutura.
Complexidade evitada	Microserviços, Kubernetes, Kafka, GraphQL, Redis e outros componentes somente se surgir necessidade comprovada.

2. Princípios arquiteturais

Simplicidade operacional: A solução deve ser simples de executar, testar, implantar e manter pela equipe atual.

Separação de responsabilidades: Frontend não concentra regras críticas; backend controla regras, autorização, transações e persistência.

Consistência de dados: Operações que alteram estoque devem ser transacionais e preservar integridade.

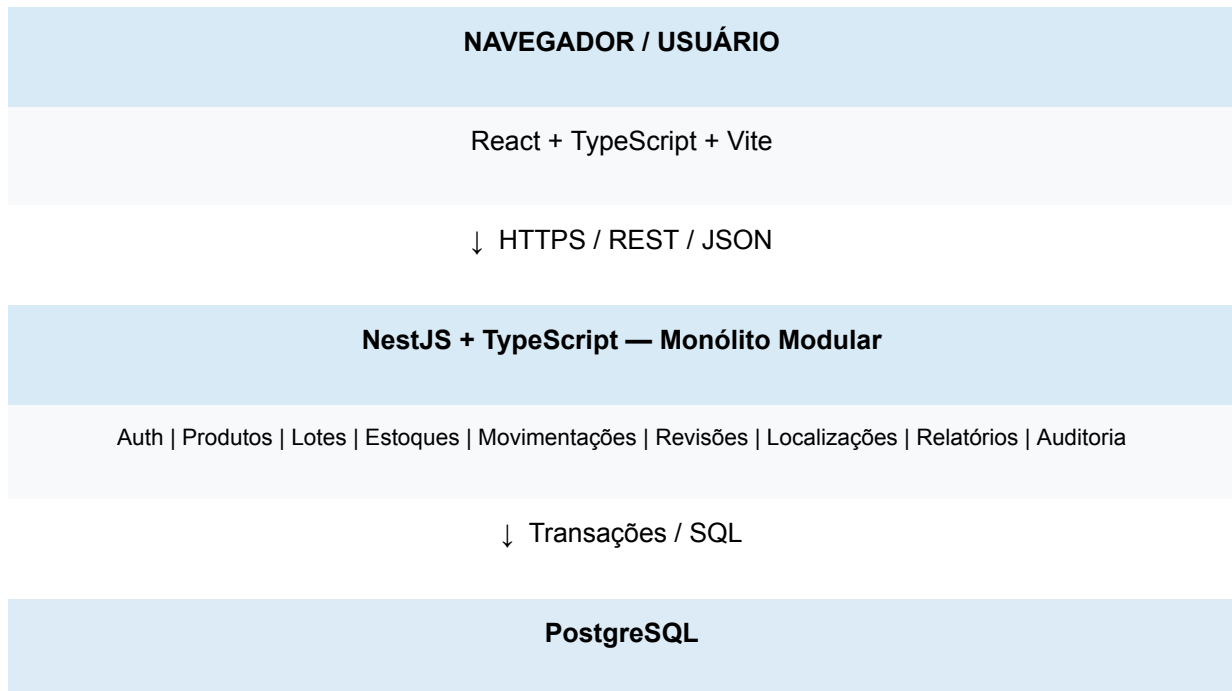
Rastreabilidade: Toda ação relevante deve ser autenticada, autorizada, registrável e auditável.

Configuração externa: Ambiente, credenciais, URLs e segredos não devem ficar fixos no código.

Evolução incremental: Recursos de infraestrutura só serão adicionados quando houver necessidade concreta.

3. Arquitetura geral

O sistema será desenvolvido como um **monólito modular**, mantendo uma única aplicação backend, mas separada internamente por domínios e responsabilidades. O frontend será uma aplicação independente que consome a API do backend.



3.1 Por que monólito modular

- Permite deploy simples como uma única aplicação backend.
- Reduz custos de infraestrutura, observabilidade e comunicação distribuída.
- Mantém módulos organizados e com responsabilidades claras.
- Permite que um módulo seja extraído futuramente, caso escala ou integração realmente exijam isso.
- É adequado ao tamanho da equipe e ao estágio atual do sistema.

Decisão registrada

Microserviços não serão utilizados na primeira arquitetura. A adoção futura somente deverá ocorrer caso exista justificativa técnica mensurável, como escalabilidade independente, isolamento operacional ou necessidade real de integração distribuída.

4. Estrutura lógica e módulos

Módulo	Responsabilidade principal
Auth	Login, sessão/token, recuperação de acesso e eventos de autenticação.
Users	Usuários, perfis, status e permissões.
Products	Cadastro e informações dos produtos.
Batches	Lotes, validade e vínculo com produtos.
Stocks	Estoques, subestoques e saldos.
Movements	Entradas, saídas, transferências e operações compensatórias.

Módulo	Responsabilidade principal
Reviews	Processo de revisão e classificações de produtos/lotos.
Locations	Representação da localização física e mapa 2D.
Reports	Consultas, filtros e relatórios operacionais.
Audit	Registro de ações e alterações sensíveis.
Config	Configurações administrativas e parâmetros do sistema.

Estrutura sugerida do backend:

```
src/
modules/
  auth/
  users/
  products/
  batches/
  stocks/
  movements/
  reviews/
  locations/
  reports/
  audit/
shared/
database/
config/
main.ts
```

5. Organização interna do backend

Os módulos deverão manter uma divisão simples de responsabilidades:

Controller HTTP / entrada	Service / Use Case Regra de negócio	Repository Persistência	PostgreSQL Dados
-------------------------------------	---	-----------------------------------	----------------------------

Controller: recebe a requisição, valida formato de entrada e devolve resposta HTTP.

Service/Use Case: executa regras de negócio, autorização contextual, transações e orquestração.

Repository: encapsula as consultas e gravações do banco.

Frontend: não deve ser fonte de verdade para saldo, autorização ou regras críticas.

6. API e comunicação

- API REST utilizando JSON.
- Versionamento desde o início: /api/v1/...

- Validação de entrada no backend por DTO/schema.
- Padronização de códigos de erro e respostas.
- HTTPS obrigatório em produção.

Exemplos de rotas:

- /api/v1/products
- /api/v1/batches
- /api/v1/stocks
- /api/v1/movements
- /api/v1/reviews
- /api/v1/reports

7. Persistência e modelo de dados

O banco principal será PostgreSQL. O sistema deverá distinguir claramente **estado atual** de **histórico de operações**.

Regra estrutural

Registros históricos de entrada, saída, transferência e revisão não devem ser reescritos apenas porque o saldo atual mudou. Histórico e saldo possuem finalidades diferentes.

7.1 Estoques configuráveis

- Estoques e subestoques serão entidades de banco e não regras fixas no código.
- Não serão criadas colunas específicas como saldo_tuf, saldo_varejo ou saldo_lata_boa.
- Novos destinos/classificações deverão poder ser adicionados sem alterar a estrutura principal da aplicação.

7.2 Lotes

- Movimentações poderão selecionar lote e quantidade.
- Operações de revisão poderão envolver mais de um produto/lote quando necessário.
- O processo poderá informar um novo lote conforme definido nas regras de negócio.

7.3 Transações

- Toda operação que afete saldo deverá executar em transação de banco.
- Movimentação, alteração de saldo e auditoria devem ser confirmadas juntas ou revertidas juntas.
- Falhas parciais deverão produzir rollback, evitando divergência entre origem, destino e histórico.

8. Movimentações e revisão

O módulo de movimentações será um núcleo transversal do sistema. Os tipos iniciais contemplados são:

- Entrada externa
- Saída externa
- Transferência interna entre estoques/subestoques
- Cancelamento, estorno ou operação compensatória quando aplicável

8.1 Cancelamento e exclusão funcional

Para o usuário poderá existir a ação de cancelar ou excluir uma revisão. Entretanto, operações que já produziram efeito no estoque não deverão desaparecer sem evidência. O backend deverá preservar o evento original e executar a reversão necessária de forma controlada.

Princípio de integridade

Nenhuma operação com impacto sobre estoque deverá ser removida de forma que impeça reconstruir o que ocorreu, quem executou e quais saldos foram afetados.

9. Localização física e mapa 2D

A posição física não será utilizada como substituta do saldo lógico. O mapa será uma representação visual vinculada aos registros do sistema.

- Saldo lógico e localização física serão conceitos independentes.
- Blocos do mapa poderão representar áreas/posições físicas.
- Produtos e lotes poderão ser associados a uma localização.
- Alterar a posição visual não deverá, por si só, criar ou alterar saldo.

10. Segurança e autenticação

Segurança será requisito de base. A autenticação não será tratada como funcionalidade opcional ou adicionada apenas no final do projeto.

10.1 Senhas

- Senhas nunca serão armazenadas em texto puro ou de forma reversível.
- Será utilizado algoritmo específico para armazenamento de senha, preferencialmente Argon2id ou bcrypt.
- Cada senha deverá utilizar salt adequado, gerado no processo de hash.
- A validação será feita por comparação segura do hash; o sistema não deverá possuir função de “descriptografar senha”.
- Parâmetros de custo do algoritmo deverão ser configurados de forma compatível com o ambiente de produção.

10.2 Sessão e acesso

- Sessões ou tokens terão expiração e política de renovação controlada.
- Logout deverá invalidar ou encerrar a sessão conforme a estratégia adotada.
- Em autenticação web baseada em cookies, utilizar HttpOnly, Secure e SameSite quando aplicável.
- Permissões devem ser validadas no backend, independentemente de botões ou telas ocultos no frontend.
- Perfis iniciais poderão ser ADMIN, OPERADOR e CONSULTA, evoluindo para permissões granulares conforme necessário.

10.3 Proteção de aplicação

- HTTPS obrigatório no ambiente produtivo.
- Credenciais, chaves e segredos fora do repositório e do código-fonte.
- Variáveis de ambiente ou mecanismo de secret management para credenciais.

- Validação e sanitização de entradas no backend.
- Tratamento de tentativas de login, falhas de autenticação e eventos de segurança.
- Princípio de menor privilégio para acesso ao banco e demais serviços.

11. Histórico, auditoria e logs

O projeto possuirá três camadas de rastreabilidade independentes. Elas não devem ser misturadas em uma única tabela ou conceito:

Camada	Responde principalmente	Exemplos
Histórico de negócio	O que aconteceu com o estoque?	Entrada, saída, transferência, revisão, classificação, estorno.
Auditoria	Quem fez o quê e o que mudou?	Usuário, ação, entidade, antes/depois, data, origem, request ID.
Log técnico	Como a aplicação se comportou?	Erro, exceção, timeout, falha de autenticação, integração, performance.

11.1 Auditoria

- Toda operação relevante deve identificar o usuário responsável.
- Registrar ação, tipo da entidade, identificador, data/hora e resultado.
- Quando aplicável, registrar valores anteriores e novos de forma estruturada.
- Poder registrar IP, user-agent e identificador de requisição conforme necessidade e política da empresa.
- Usuários comuns não poderão editar ou excluir auditorias.
- Mesmo administradores funcionais não deverão possuir, por padrão, permissão para apagar evidências de auditoria.

Estrutura conceitual sugerida:

```
audit_log
id
user_id
action
entity_type
entity_id
old_values
new_values
ip_address
user_agent
request_id
created_at
```

11.2 Logs técnicos

- Logs deverão ser estruturados, preferencialmente em formato que permita pesquisa e centralização futura.
- Não registrar senhas, tokens completos, segredos ou dados sensíveis desnecessários.
- Erros deverão possuir correlação com request ID para facilitar diagnóstico.
- A estratégia de retenção e centralização poderá evoluir conforme o ambiente de produção.

12. Arquivos e anexos

Fotos ou anexos não deverão ser gravados como arquivo bruto dentro das tabelas principais do PostgreSQL. O banco armazenará metadados e referência ao armazenamento.

- Desenvolvimento/servidor simples: filesystem controlado.
- Evolução: armazenamento compatível com S3, Azure Blob, MinIO ou equivalente.
- A regra de negócio não deverá depender diretamente do provedor escolhido.

13. Ambientes e implantação

13.1 Desenvolvimento

- Frontend em execução local (ex.: porta 5173).
- Backend em execução local (ex.: porta 3000).
- PostgreSQL instalado diretamente na máquina durante a fase inicial, conforme decisão do projeto.
- Docker não será obrigatório para desenvolver localmente.

13.2 Produção em servidor da empresa

A implantação recomendada deverá utilizar contêineres quando o sistema for para produção, mantendo uma composição simples:

- Frontend
- Backend
- PostgreSQL ou instância PostgreSQL externa
- Nginx/reverse proxy
- HTTPS/certificado
- Persistência e backup

Estratégia de deploy

Docker e Docker Compose serão usados como estratégia de empacotamento e implantação quando trouxerem benefício operacional, sem obrigar a equipe a utilizá-los na primeira fase de desenvolvimento.

13.3 Nuvem

A arquitetura deverá permitir migração futura para nuvem sem reescrever o domínio. Os principais componentes podem ser substituídos por serviços gerenciados de infraestrutura: aplicação, banco PostgreSQL, armazenamento de arquivos e observabilidade.

14. Configuração e portabilidade

O código não deverá depender de endereços, credenciais ou caminhos fixos. Parâmetros de ambiente serão externos ao código.

- DATABASE_URL
- JWT/SESSION secrets
- PORT
- FRONTEND_URL / API_URL

- FILE_STORAGE_PATH ou configuração de object storage
- Ambiente (development, test, production)
- Configurações de logging

15. Operação, backup e observabilidade

- Backup do PostgreSQL fará parte da estratégia de produção.
- Arquivos anexados também devem possuir estratégia de backup quando utilizados.
- Logs de aplicação deverão permitir diagnóstico de incidentes.
- Auditoria deverá possuir retenção compatível com a política definida pela empresa.
- Métricas avançadas e centralização de logs poderão ser incorporadas posteriormente sem alterar o domínio.

16. Tecnologias deliberadamente fora da primeira versão

Os itens abaixo não estão proibidos no futuro. A decisão é apenas não introduzi-los antes de existir necessidade técnica concreta:

- Microserviços
- Kubernetes
- Kafka
- RabbitMQ
- Redis sem caso de uso comprovado
- Elasticsearch
- GraphQL
- CQRS completo
- Event Sourcing completo
- Múltiplos bancos por domínio
- API Gateway dedicado
- Autenticação corporativa/Active Directory na primeira versão

17. Requisitos não funcionais derivados

Característica	Diretriz
Segurança	Acesso autenticado, autorização no backend, proteção de credenciais e transporte criptografado.
Integridade	Operações críticas transacionais e validação de saldo/lote/origem/destino.
Auditabilidade	Reconstrução de ações relevantes e identificação do responsável.
Manutenibilidade	Código modular, responsabilidades claras e baixo acoplamento entre domínio e infraestrutura.
Portabilidade	Implantação possível em máquina local, servidor corporativo ou nuvem.

Característica	Diretriz
Evolução	Módulos e infraestrutura poderão crescer incrementalmente.
Recuperação	Backups e estratégia de restauração no ambiente produtivo.
Observabilidade	Logs técnicos e identificação de requisições/erros para diagnóstico.

18. Registro consolidado de decisões arquiteturais

ID	Tema	Decisão	Status
ADR-01	Arquitetura	Monólito modular	Aprovada
ADR-02	Frontend	React + TypeScript + Vite	Aprovada
ADR-03	Backend	Node.js + TypeScript + NestJS	Aprovada
ADR-04	API	REST/JSON versionada	Aprovada
ADR-05	Persistência	PostgreSQL	Aprovada
ADR-06	Autenticação	Hash seguro de senha + sessão/token protegido	Aprovada
ADR-07	Autorização	Perfis/permisões validados no backend	Aprovada
ADR-08	Auditoria	Auditoria independente do histórico funcional	Aprovada
ADR-09	Logging	Logs técnicos estruturados	Aprovada
ADR-10	Movimentações	Histórico preservado e operações transacionais	Aprovada
ADR-11	Estoques	Entidades configuráveis, sem hardcode	Aprovada
ADR-12	Localização	Mapa físico separado do saldo lógico	Aprovada
ADR-13	Arquivos	Referência no banco + storage externo/filesystem	Aprovada
ADR-14	Deploy	Local simples; Docker Compose na produção quando aplicável	Aprovada
ADR-15	Infraestrutura	Compatível com servidor da empresa ou cloud	Aprovada

19. Próximos artefatos técnicos

Com esta arquitetura registrada, os próximos documentos/modelos devem ser produzidos a partir destas decisões, sem redefinir a base estrutural a cada etapa:

1. Modelo de domínio e limites de cada módulo.
2. Modelagem conceitual e lógica do banco de dados.
3. Definição detalhada das entidades e relacionamentos.
4. Contrato inicial da API REST.
5. Matriz de perfis e permissões.
6. Regras de auditoria por tipo de operação.
7. Estratégia de testes (unitário, integração e fluxo crítico).
8. Plano de implantação e backup quando o ambiente produtivo for definido.

Diretriz final aprovada

Toda operação relevante deverá ser autenticada, autorizada, rastreável e auditável. Nenhuma senha será armazenada de forma reversível, e nenhuma operação com impacto sobre estoque deverá ser removida sem preservar evidência suficiente para reconstrução do ocorrido.

Registro

Este documento representa a arquitetura-base aprovada para orientar a implementação inicial do sistema. Mudanças relevantes deverão ser registradas como novas decisões arquiteturais, preservando o motivo e o impacto da alteração.